

Worksheet · Section 3.9–3.10

Developing Algorithms & Lists



The Big Picture

Classic algorithms show up on every AP exam: max, min, average, search, filter. Practice list procedures, robot moves, and building algorithms from parts.

Part 1 · Vocab Match

Write the letter of the matching definition on each line.

- | | |
|---------------------------------------|--|
| _____ 1. Algorithm | A. Building a new list of only the elements that meet a condition |
| _____ 2. Linear search | B. Returns true if the robot can move to the square in that direction |
| _____ 3. Traversal | C. An existing, correct algorithm reused in a new one |
| _____ 4. APPEND | D. Deletes the element at an index and shifts the rest left |
| _____ 5. INSERT | E. Checking each element in order until the target is found |
| _____ 6. REMOVE | F. Returns the number of elements in a list |
| _____ 7. LENGTH | G. Turns the robot 90° counterclockwise |
| _____ 8. Equivalent algorithms | H. Algorithms that give the same output for every input |
| _____ 9. ROTATE_LEFT | I. Visiting the elements of a list one at a time |
| _____ 10. CAN_MOVE | J. Puts a value at an index and shifts the rest right |
| _____ 11. Filter | K. A finite set of instructions that accomplish a task |
| _____ 12. Building block | L. Adds a value to the end of a list |

Part 2 · Fill It In

Use each word from the word bank once.

MOVE_FORWARD**left****90****right****equivalent****first****LENGTH****end**

1. APPEND adds a value to the _____ of a list.
2. INSERT shifts later elements to the _____.
3. REMOVE shifts later elements to the _____.
4. _____ returns the number of elements in a list.
5. To find the max, start with the _____ element, not 0.
6. _____ moves the robot one square in the direction it faces.
7. ROTATE_RIGHT turns the robot _____ degrees clockwise.
8. Two algorithms that always produce the same output are _____.

Worksheet · Section 3.9–3.10

Practice

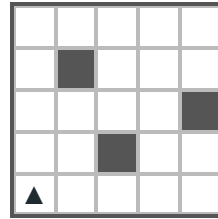
Part 3 · List Operations

`q ← [5, 10, 15]`. Write `q` after each line.

Line	q
<code>APPEND(q, 20)</code>	_____
<code>INSERT(q, 1, 0)</code>	_____
<code>REMOVE(q, 3)</code>	_____
<code>q[2] ← q[4]</code>	_____

`LENGTH(q)` = _____

Part 4 · Robot Moves



▲ = robot facing up
dark = wall

```
REPEAT 2 TIMES
{
  MOVE_FORWARD()
  MOVE_FORWARD()
  ROTATE_RIGHT()
}
```

The robot ends in row _____ (from the top), column _____, facing _____. Draw its path on the grid.

Part 5 · Classic Algorithms

```
PROCEDURE average(nums)
{
  total ← _____
  FOR EACH n IN nums
  { total ← total + _____ }
  RETURN (total / _____)
}
```

```
PROCEDURE findMin(nums)
{
  min ← nums[_____]
  FOR EACH n IN nums
  {
    IF (n _____ min)
    { min ← n }
  }
  RETURN (min)
}
```

Part 6 · Filter It

Write code that builds a new list `big`s containing every value in `scores` greater than 50. Then give `big`s for `[40, 75, 50, 90]`.



Think:

Max/min trap: always start with the list's first element. Starting at 0 breaks on lists of all-negative (or all-large positive) numbers.

Worksheet · Section 3.9–3.10

Exam Practice

Part 7 · Exam Practice

★ Warm-up

1. What is `lst`?

```
lst ← [4, 8]
INSERT(lst, 2, 6)
```

- A. [4, 6, 8]
- B. [4, 2, 8]
- C. [6, 4, 8]
- D. [4, 8, 6]

★★ On Level

2. A robot faces right and runs `ROTATE_LEFT()` three times. Which way does it face?

- A. Right
- B. Left
- C. Down
- D. Up

★★ On Level

3. What does this code do?

```
c ← 0
FOR EACH x IN lst
{
  IF (x < 0)
  { c ← c + 1 }
}
```

- A. Adds the negative values
- B. Finds the smallest value
- C. Counts the negative values
- D. Removes negative values

★★★ Exam Hard

4. What is `lst` afterward?

```
lst ← [1, 2, 3, 4, 5]
i ← 1
REPEAT UNTIL (i > LENGTH(lst))
{
  IF (lst[i] MOD 2 = 0)
  { REMOVE(lst, i) }
  ELSE
  { i ← i + 1 }
}
```

- A. [2, 4]
- B. [1, 3, 5]
- C. [1, 3, 4, 5]
- D. [1, 2, 3, 4, 5]

★★★ Exam Hard

5. Which TWO segments correctly compute the sum of `lst`? (*Select two.*)

- A. `s ← 0`, then `FOR EACH x IN lst { s ← s + x }`
- B. `s ← 0`, `i ← 1`, then `REPEAT LENGTH(lst) TIMES { s ← s + lst[i] i ← i + 1 }`
- C. `s ← 0`, then `FOR EACH x IN lst { s ← x }`
- D. `s ← lst[1]`, then `FOR EACH x IN lst { s ← s + x }`

Part 8 · Written Response

Describe in plain English an algorithm that finds the second-largest value in a list. Then describe two test lists you would use to check it.
